

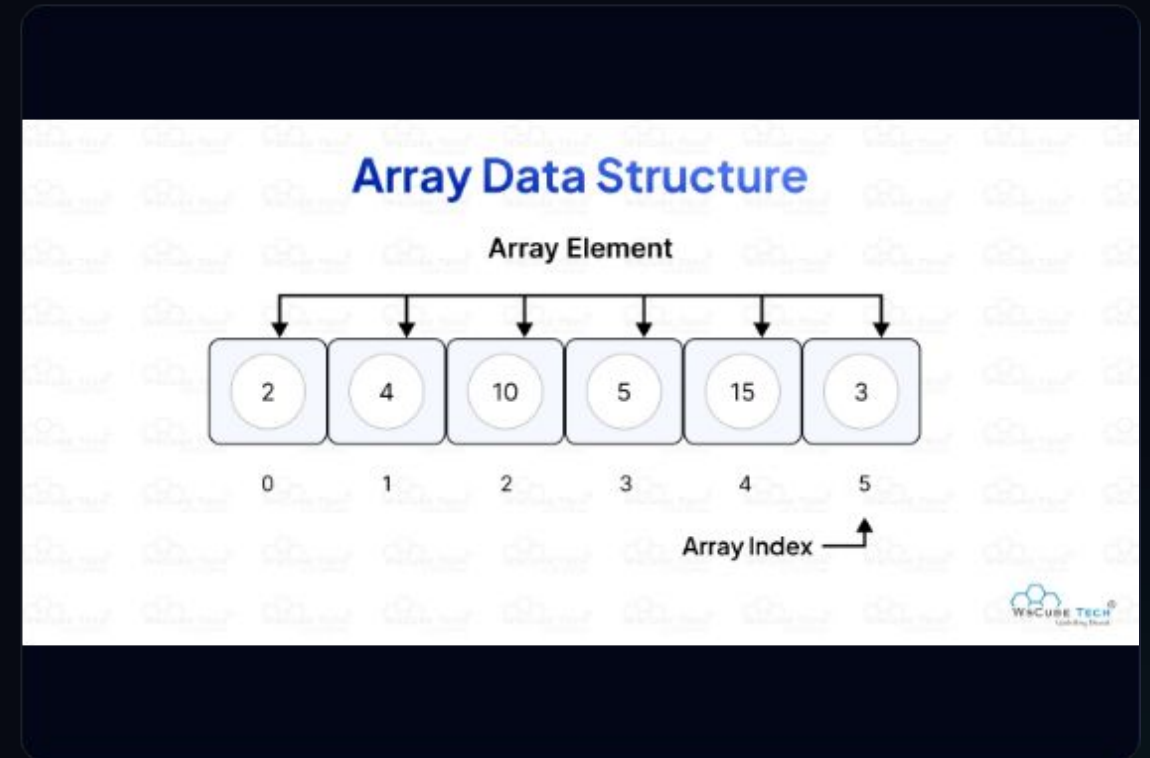
MÓDULO 1 · FUNDAMENTOS DE C#

Estructuras de Datos: Arrays

Introducción a la programación con .NET Core y gestión de colecciones indexadas de datos uniformes de alto rendimiento.

| ¿Qué es un Array o Vector?

- 📦 **Colección homogénea:** Estructura capaz de almacenar múltiples valores que pertenecen estrictamente al mismo tipo de datos.
- 📍 **Acceso indexado:** Se accede a cada elemento directamente mediante su posición de índice lógica.
- 🔧 **Declaración básica:** Indicamos el tipo seguido de corchetes vacíos:
`int[] numeros;`



Inicialización y Acceso

1. Reserva de Memoria

Uso de la instrucción `new` indicando explícitamente el tamaño fijo entre corchetes.

```
// Reserva memoria para 5 enteros  
int[] nums = new int[5];  
nums[0] = 12; // Asignación
```

2. Asignación Directa (C# 12)

Si se conocen los valores al inicio, se puede usar la nueva y concisa sintaxis de expresión de colección.

```
// Sintaxis moderna basada en corchetes  
int[] nums = [12, 23, 53, 5, 92];
```

| Recorrer un Array de Datos

Bucle clásico (for)

Ideal cuando requerimos manipular la variable de control del índice durante el recorrido completo.

```
for (int i = 0; i < nums.Length; i++)  
{  
    Console.WriteLine($"Índice {i}: {nums[i]}");  
}
```

Bucle de lectura (foreach)

Más limpio, rápido y seguro. Ideal para leer valores secuencialmente sin manipular índices.

```
foreach (int num in nums)  
{  
    Console.WriteLine(num);  
}
```

Búsqueda de Elementos

Búsqueda Manual

Rastreo secuencial comparando elemento a elemento, deteniendo el bucle al hallar el elemento.

```
bool encontrado = false;
for (int i = 0; i < nombres.Length && !encontrado; i++)
{
    if (nombres[i] == "Pedro")
        encontrado = true;
}
```

Helper Nativo (.NET API)

La clase estática Array posee utilidades optimizadas con expresiones de predicado.

```
// Búsqueda interna directa
bool encontrado = Array.Exists(nombres,
    n => n == "Pedro");
```

| Redimensionar Arrays en C#

- ❗ **Limitación de tamaño:** Los arrays son de tamaño estático asignado en memoria.
- 📄 **Estrategia de copia:** Se requiere inicializar un nuevo array auxiliar mayor, copiar los elementos y reasignarlo.
- ⚡ **Función automatizada:** .NET abstrae este proceso de forma transparente mediante la API Resize.

Uso de Array.Resize

La referencia se pasa por parámetro utilizando el modificador de referencia ref.

```
// Modifica la memoria asignada
Array.Resize(ref nombres, 10);
nombres[5] = "Marta";
// nombres.Length ahora devolverá 10
```

Borrar un Elemento del Array

Desplazamiento de Ítem

Para no dejar celdas intermedias vacías, desplazamos a la izquierda todos los elementos situados a la derecha del ítem borrado.

```
// Borramos la posición índice 2
for (int i = 2; i < guardados - 1; i++)
{
    nums[i] = nums[i + 1];
}
guardados--; // Se actualiza contador
```

Borrado con Resize

Ajustamos físicamente el tamaño del array disminuyendo un elemento tras desplazar la información.

```
// Desplazamos y reducimos la memoria
Array.Resize(ref nums, nums.Length - 1);
```

Añadir o Insertar Elementos

Desplazamiento Inverso

Al insertar en una posición intermedia, se debe liberar la celda recorriendo el array al revés para desplazar los elementos a la derecha.

- Primero redimensionamos la memoria para asegurar espacio.
- Movemos desde el final hacia el índice destino.

Sintaxis en C#

```
// Redimensiona un elemento más  
Array.Resize(ref nums, nums.Length + 1);  
  
// Desplazamiento preventivo a la derecha  
for (int i = nums.Length - 2; i ≥ 3; i--) {  
    nums[i + 1] = nums[i];  
}  
nums[3] = 2; // Inserta el 2
```


| Ordenación de Colecciones

Algoritmo de Burbuja

Estructura didáctica iterativa que compara pares de índices intercambiando valores de menor valor al inicio.

```
if (nums[j] < nums[i])  
{  
    int aux = nums[i];  
    nums[i] = nums[j];  
    nums[j] = aux;  
}
```

Ordenación Nativa

.NET optimiza drásticamente la ordenación interna a través del método ultra-rápido de la clase Array.

```
// Ordena en orden ascendente in-place  
Array.Sort(nums);  
// Rendimiento óptimo garantizado
```

| Operadores de Rango y Spread



Sintaxis de Rangos

Genera un subarray especificando límites mediante los dos puntos (..).

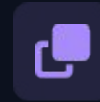
```
int[] parte = num[2..5];
```



Patrones de Lista

Evalúa de manera inteligente si un array coincide con una estructura fija usando is.

```
if (nums is [1, 2, var resto])
```



Operador Spread

Utiliza la desestructuración de colecciones para concatenar o clonar de manera veloz.

```
int[] copia = [..nums];
```

Matrices Multidimensionales

1. Array Multidimensional Rectangular

Dimensiones de tamaño uniforme. Se declara indicando comas internas entre los corchetes.

```
int[,] array2D = new int[4, 3];  
// Inicialización  
int[,] matrix = { {1,2}, {3,4} };
```

2. Array de Arrays (Escalonado)

Arrays cuyas filas tienen dimensiones de longitudes independientes y variables (*Jagged Arrays*).

```
int[][] arrayJagged = new int[4][];  
arrayJagged[0] = new int[3];  
arrayJagged[1] = new int[5]; // Diferente
```